

may not be playable.

```
echo "file 'tank-cut-recoded.mp4'" >
file-list.txt
echo "file 'tank.mp4'" >> file-list.txt
ffmpeg -f concat -i file-list.txt -c
copy tank-mix.mp4
```

When you try to individually convert the input videos down to a common codec, dimensions and frame rate, and then convert them all into a single file, the lossy algorithm does damage twice. To limit the loss, it is best if you use the raw uncompressed video of each input video. This is rarely the case.

Convert to MP4

While I have no problems with using Ogg video on my Linux systems, my electronic devices do not support it. (This is ironic because they run on Linux.) As I am unable to stream videos because of poor Internet connectivity, I have to convert all non-MP4 videos. The conversion I had suggested in the original article was based on constant bit rate. A better goal is constant quality. FFmpeg's native x264 codec has several presets, ranging from 'veryslow' to 'ultrafast'. You also need to use the constant rate factor option 'crf', whose value ranges from 0 (lossless) to 63 (worst).

```
ffmpeg -i elastigirl.mp4 \
-c:v libx264 -crf 31 -preset
slow \
-c:a copy \
incredible.mp4
```

x264 Presets	x264 Tune	x264 Profiles
ultrafast	film	baseline
superfast	animation	main
veryfast	grain	high
faster	stillimage	
fast	psnr	
medium	ssim	
slow	fastdecode	
slower	zerolatency	
veryslow		
placebo		

Figure 2: Presets available for the x264 codec

Although FFmpeg continues to support several other x264 codecs, the native encoder (libx264) is now the preferred option. An advantage with constant quality is that multiple passes are not required (used to achieve the correct bit rate).

Cut a video

When a raw uncompressed video is converted using a video codec, the compression algorithm creates keyframes to eliminate redundant data. When you cut a video at a particular timestamp, the frames near that location may not have all the information because they are not keyframes. (In other words, clean cuts are not possible with highly compressed videos.) Some frames will have to be eliminated and 'video jumps' become noticeable. The audio and video may also go out of sync.

To limit these problems, whenever you cut a video, fix the timestamps of the frames before further processing.

```
ffmpeg -y -i barbara.mp4 \
-filter_complex \
"[0:v:0]trim=start=0:end=16,
setpts=PTS-STARTPTS[lv];
[0:v:0]trim=start=36:end=44,
setpts=PTS-STARTPTS[rv];
[0:a:0]atrim=start=0:end=16,
asetpts=N/SAMPLE_RATE/TB[la];
[0:a:0]atrim=start=36:end=44,
asetpts=N/SAMPLE_RATE/TB[ra];
[lv][rv]concat=n=2:v=1:a=0[v];
[la][ra]concat=n=2:v=0:a=1[a]" \
-map '[v]' -map '[a]' barb-cut.mp4
```

In this example, the audio and video from seconds 16 to 44 are removed, and the timestamps of the remaining segments are counted again before being provided to the concat filters.

Rotate a video

In the original article, you learned that a video can be rotated in spans of 90°. Values 0 and 3 flip the video in addition to the rotation.

```
ffmpeg -i barb.mp4 \
-filter:video "transpose=1" \
barb-rotated.mp4
```

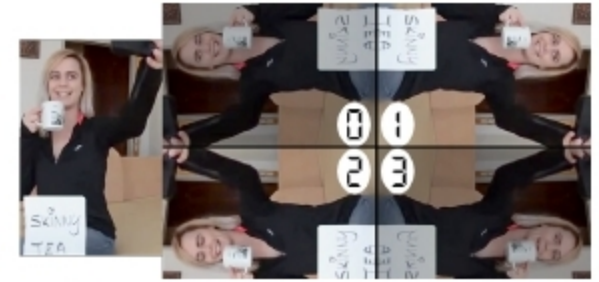


Figure 3: Rotate filter options

Figure 3 shows the transpose filter; filter values 1 and 2 rotate the video right and left. The values 0 and 3 cause rotation but also flip the video.

More discrete levels of rotation are possible with the rotate filter.

```
ffmpeg -y -i barbara.mp4 \
-vf rotate=angle=-
20*PI/180:fillcolor=brown
barb20.mp4
```



Figure 4: This video has been tilted to the left by 20°

If you want to simply flip the video, use `vflip` and `hflip` filters.

```
ffmpeg -i lucas.mp4 \
-filter:v "vflip" \
lucas-upside-down.mp4
ffmpeg -i lucas.mp4 \
-filter:v "hflip" \
lucas-half-crazy.mp4
ffmpeg -i lucas.mp4 \
-filter:v "hflip,vflip" \
lucas-totally-flipped.mp4
```



Figure 5: The flip filters are used to horizontally and vertically flip a video